The main purpose of exception handling is to provide user-friendly message so, client can enjoy the services of software/website.

Exception handling → **No Abnormal Termination + User-friendly message on wrong input given by the client**

```
package com.rwd.basic;

public class CustomerDemo
{
    public static void main(String[] args)
    {
        clientRequest();
    }

    public static void clientRequest()
    {
        System.out.println("Hello Client, Welcome to my Application");

        Try
        {
            int x = Integer.parseInt(BufferedReader.readLn("Enter the value of x :"));
            int y = Integer.parseInt(BufferedReader.readLn("Enter the value of y :"));

            int result = x / y;
            System.out.println("Result is :"+result);
        }
        catch(Exception e)
        {
            System.err.println("Please don't put zero here");
        }

        System.out.println("Thank you for visiting my Application");
    }
}
```

Throwable class methods to print Exception :

Throwable class has provided the following three non static methods :

1) public String getMessage() : It will provide only error message.

2) public void printStackTrace() : It will provide the complete details regarding exception like exception class name, exception error message, exception class location, exception method name and exception line number.

3) public String toString() : It will convert the exception into String representation.

:Program:

```
package com.rwd.basic;

public class ThrowableMethods {

    public static void main(String[] args)
    {
        Try
        {
            int[]arr = {11, 88, 55, 78};
            System.out.println(arr[7]);
        }

        catch(Exception e)
        {
            System.out.println(e.toString());//Type : Error+Message
            System.out.println("-------------------------------");
            System.out.println(e.getMessage());//Error+Message
            System.out.println("-------------------------------");
            e.printStackTrace();//Complete exception details
        }

    }
}
```

Working with Specific Exception :

While working with exception, in the corresponding catch block we can take Exception (super class) which can handle any type of Exception.

On the other hand we can also take specific type of exception (ArithmeticException, InputMismatchException and so on) which will handle only one type i.e specific type of exception.

:Program :

```
package com.rwd.basic;

import java.util.Arrays;

public class SpecificException
{
    public static void main(String[] args)
    {
        Try
        {
            int size = Integer.parseInt(BufferedReader.readLn("Enter the size of the array :"));
            int[] arr = new int[size];
            for(int i : arr)
            {
                System.out.print(0);
            }
            System.out.println();
        }
        catch(NegativeArraySizeException e)
        {
            System.err.println("Array size must be a positive Integer");
        }

        System.out.println("Main method completed");
    }
}


package com.rwd.basic;

public class SpecificExceptionDemo2
{
    public static void main(String[] args)
    {
        Try
        {
            throw new OutOfMemoryError();
        }
        catch(Error e)
        {
            System.err.println("Out of memory error");
        }

    }
}
```

Working with multiple try catch :

According to our application requirement we can provide **multiple try-catch** in his application to work with different exceptions.

:Program:

```
package com.rwd.basic;

public class MultiTryCatch {

    public static void main(String[] args)
    {
        Try
        {
            String str = null;
            System.out.println(str);
        }
        catch(NullPointerException e)
        {
            System.err.println("reference variable is pointing to null");
        }

        Try
        {
            String s1 = "java";
            Integer val = Integer.valueOf(s1);
            System.out.println(val);
        }
        catch(NumberFormatException e)
        {
            System.err.println("Wrong input is not in a proper format");
        }

        Try
        {
            Object[] arr = new String[3];
            arr[0] = "Sai";
            arr[1] = "Ram";
            arr[2] = 786;
        }
        catch(ArrayStoreException e)
        {
            System.err.println("Cannot insert illegal data into array");
        }
    }
}
```

Note : In the above program the standard is, Client will get all the error messages at once which is not a recommended way.

In order to avoid this drawback we introduced multiple catch blocks with single try.

**** Single try with multiple catch blocks :**

```
Try
{
    int[]
    arr=0];
    System.out.println(arr[7]);
}
catch(ArrayIndexOutOfBoundsException e)
{
    System.err.println("Array index is out of bounds");
}
catch(ArithmeticException e)
{
    System.err.println("Divide by zero problem");
}
catch(Exception e)
{
    System.err.println("General Problem");
}
```

Try
{
int[]
arr=0];
System.out.println(arr[7]);
}
catch(Exception e)
{
System.err.println("Divide by zero problem");
}

Cannot provide appropriate Exception message

According to industry standard we should write try with multiple catch blocks so we can provide proper information for each and every exception to the end user.

While working with multiple catch block always the super class catch block must be last catch block. [Unreachable code]

From java 1.7v this multiple exceptions we can write in a single catch block by using | symbol.

Example :

```
Try
{
}
catch(ArrayIndexOutOfBoundsException | NullPointerException e)
{
}
```

If try block is having more than one exception then always try block will enterface only first exception because control will transfer to the nearest catch block.

:Program :

```
package com.rwd.basic;

public class MultipleCatch {

    public static void main(String[] args)
    {
        Try
        {
            int i = 10 [0];
            System.out.println(i);
            int[]arr = {10,20,30,40};
            System.out.println(arr[5]);
        }

        catch(ArrayIndexOutOfBoundsException e)
        {
            System.err.println("Array index is out of bounds");
        }
        catch(ArithmeticException e)
        {
            System.err.println("Divide by zero problem");
        }
        catch(Exception e)
        {
            System.err.println("General Problem");
        }

    }
}
```

Another program by using | symbol.

```
package com.rwd.basic;

public class MultipleCatchBlocks {

    public static void main(String[] args)
    {
        System.out.println("Main method started!!");
        Try
        {
            String str1 = "null";
            System.out.println(System.getProperties());//NULL

            String str2 = "java";
            int val = Integer.valueOf(str2);
            System.out.println("Number is :"+val);

            catch(NumberFormatException | NullPointerException  e)
            {
            if(e instanceof NumberFormatException)
            {
                System.err.println("Number is not in a proper format");
            }
            else if(e instanceof NullPointerException)
            {
                System.err.println("ref variable is pointing to null");
            }
            }
        }

        System.out.println("Main method ended!!");
    }

}
```